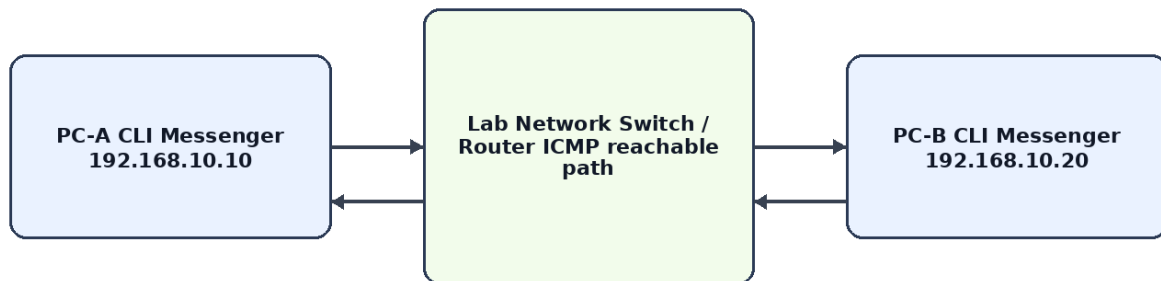


Technical Whitepaper

Network Steganography Messenger v2

ICMP Echo Transport with IPv4 Identification Field Carrier and ChaCha20-Poly1305 Encryption

Two-node authorized lab topology



Both endpoints run the same program: sender thread + receiver/sniffer thread + (

Figure 1 - Two-node authorized lab topology

Scope statement: *This document describes an authorized lab/security-research design. It is intended for controlled education, protocol engineering, packet analysis, and blue-team validation. It should not be used on networks where you do not have explicit authorization.*

Date: 2026-06-08

Table of Contents

1. Executive Summary
2. Foundations: Steganography and Network Steganography
3. Protocol Background: IPv4, ICMP, and the IPv4 Identification Field
4. Project Goals and Constraints
5. Version Selection: Why Version 2
6. High-Level Architecture
7. Lab Topology and Runtime Model
8. Cryptographic Design
9. Packet Format and Field Allocation
10. Message Lifecycle: Sender to Receiver
11. CLI Messenger Concept and User Experience
12. Control Commands and Session Management
13. Reliability Model: START, DATA, END, ACK, NACK
14. Receiver State Machine and Reassembly
15. Operational Considerations and Failure Modes
16. Defensive Analysis and Blue-Team Validation
17. Implementation Roadmap
18. References

1. Executive Summary

This whitepaper defines the design of a two-node command-line network steganography messenger for authorized lab research. The chosen design, called Version 2, uses ICMP Echo packets as a transport shell and encodes encrypted message fragments inside the 16-bit IPv4 Identification field. The messenger is intended to behave like a simple CLI chat environment between two computers while also exposing packet-level concepts for network security learning and defensive analysis.

The project combines three major domains: network protocol engineering, applied cryptography, and security monitoring. The sender takes a user message, wraps it in an internal message envelope, encrypts it using an authenticated encryption algorithm, fragments the resulting bytes into two-byte chunks, and maps each chunk into the IPv4 Identification field of an ICMP packet. The receiver captures matching ICMP packets, reorders fragments using ICMP sequence metadata, verifies the cryptographic authentication tag, decrypts the message, and displays it in the CLI as a peer chat message.

- Carrier protocol: ICMP Echo Request/Reply over IPv4.
- Covert field: IPv4 Identification field, 16 bits per packet.
- Ordering field: ICMP Sequence Number.
- Session field: ICMP Identifier.
- Cryptography: password-derived symmetric key with ChaCha20-Poly1305 AEAD.
- First UI target: continuous two-node CLI chat, similar to a minimal terminal messenger.
- Defensive requirement: capture, measure, and document observable artifacts for blue-team validation.

2. Foundations: Steganography and Network Steganography

Steganography is the practice of hiding the existence of information inside another carrier. In classical digital steganography, the carrier may be an image, audio file, video stream, or text document. In network steganography, the carrier is a network communication process: packet headers, protocol metadata, payload structures, packet timing, retransmission behavior, or ordering patterns.

2.1 Steganography vs. Cryptography

Cryptography protects the meaning of a message. Steganography attempts to hide the existence or placement of a message. In this project, both are combined. The message is encrypted first, then the encrypted bytes are embedded into packet fields. This avoids the dangerous design mistake of relying on hiding alone for confidentiality.

Area	Cryptography	Steganography
Primary objective	Protect message content	Conceal message presence or carrier mapping
Failure mode	Ciphertext is visible but unreadable	Carrier is discovered and extraction method may be analyzed
Used in this project	ChaCha20-Poly1305 AEAD	IPv4-ID field chunk carrier
Best practice	Use modern authenticated encryption	Assume discovery is possible and design defensive detection

2.2 Network Steganography Categories

Category	Examples	Project relevance
Storage/header channels	IPv4 ID, TCP sequence, UDP source port, ICMP identifier	Version 2 uses IPv4 ID
Payload channels	Application data, ICMP payload, HTTP headers	Avoided as the primary secret carrier in Version 2
Timing channels	Inter-packet delay, jitter patterns	Deferred to future research
Ordering channels	Packet ordering, duplicate ordering	Not used in first version
Error/retransmission channels	Intentional loss, retransmission patterns	Not used in first version

Important engineering principle: the first implementation should be understandable and measurable before it becomes complex. Version 2 is selected because it is more protocol-focused than simple payload embedding, but less complex than a hybrid timing/header/payload system.

3. Protocol Background: IPv4, ICMP, and the IPv4 Identification Field

3.1 IPv4 Identification Field

IPv4 includes a 16-bit Identification field originally used to help reassemble fragmented datagrams. RFC 791 defines the field as an identifying value assigned by the sender to aid assembly of fragments. Later operational guidance, including RFC 6864, clarified that the IPv4 ID is meaningfully required for fragmented traffic and that modern stacks may handle the field differently for non-fragmented datagrams. This matters because a covert channel based on IPv4-ID depends on whether the field is preserved end-to-end.

- Width: 16 bits, equivalent to 2 bytes per packet.
- Normal purpose: fragment reassembly correlation.
- Project usage: carrier for one encrypted two-byte fragment per packet.
- Operational risk: middleboxes, tunnels, operating systems, or normalization devices may rewrite or alter the field.

3.2 ICMP Echo

ICMP Echo Request and Echo Reply are widely known through the ping utility. RFC 792 defines the Echo message and states that the Identifier and Sequence Number may be used by the echo sender to match replies with requests. This makes ICMP Echo useful for a lab messenger design because the ICMP Identifier can map naturally to a session identifier and the ICMP Sequence Number can map naturally to packet ordering.

ICMP field	Normal role	Messenger role
Type	Echo Request or Echo Reply	Transport shell direction
Code	Usually zero for Echo	No special meaning
Identifier	Match request/reply sessions	Session ID
Sequence Number	Incremented request number	START/DATA/END ordering
Payload	Optional data	Fixed lab payload or non-secret control bytes

Version 2 packet field usage

IPv4 Header

Version/IHL	DSCP/ECN	Total Length	Identification HIDDEN 16-bit chunk	Flags/FragOff	TTL	Protocol=ICMP	Header Checksum
-------------	----------	--------------	------------------------------------	---------------	-----	---------------	-----------------

ICMP Echo Header + Payload

Type=8/0	Code=0	Checksum	Identifier Session ID	Sequence START/DATA/END order	Payload Fixed cover/control bytes
----------	--------	----------	-----------------------	-------------------------------	-----------------------------------

Design principle: secret content is not placed in the ICMP payload. The encrypted content is carried two bytes at a time through the IPv4 Identification field. ICMP Identifier and Sequence provide session and ordering metadata.

Figure 2 - Version 2 packet field allocation

4. Project Goals and Constraints

Goal	Explanation
Two-node continuous chat	Both computers can send and receive during the same program session.
CLI first, GUI later	The first interface is a terminal chat loop; later GUI can reuse the protocol modules.
Encryption before embedding	Plaintext is never inserted into packet fields directly.
Sequencing and reassembly	Messages must survive packet reordering and basic loss scenarios.
Lab authorization	The design is for controlled testing and presentation, not unauthorized network use.
Defensive measurability	The project must produce PCAPs and analysis outputs so blue teams can study the artifact.

4.1 Non-Goals for the First Release

- No multi-user group chat.
- No automatic peer discovery.
- No file transfer in the first release.
- No GUI in the first release.
- No attempt to bypass monitoring controls; defensive measurement is included instead.
- No custom cryptographic algorithm implementation.

5. Version Selection: Why Version 2

Version	Carrier	Difficulty	Why not selected first
Version 1	ICMP payload carries encrypted chunks	Medium	Too basic for the desired protocol-level learning goal.
Version 2	IPv4 ID carries encrypted 2-byte chunks	High	Selected: strong learning value and manageable complexity.

Version 3	Hybrid header/timing/payload system	Very high	Too complex for initial implementation and debugging.
-----------	-------------------------------------	-----------	---

Version 2 is the best intermediate design because it forces the project to handle real protocol limitations: only two hidden bytes per packet, sequencing across multiple packets, packet loss, field preservation, and receiver-side reconstruction. It is advanced enough for a meaningful security project while remaining realistic for a Python implementation using packet-crafting and packet-sniffing libraries.

6. High-Level Architecture

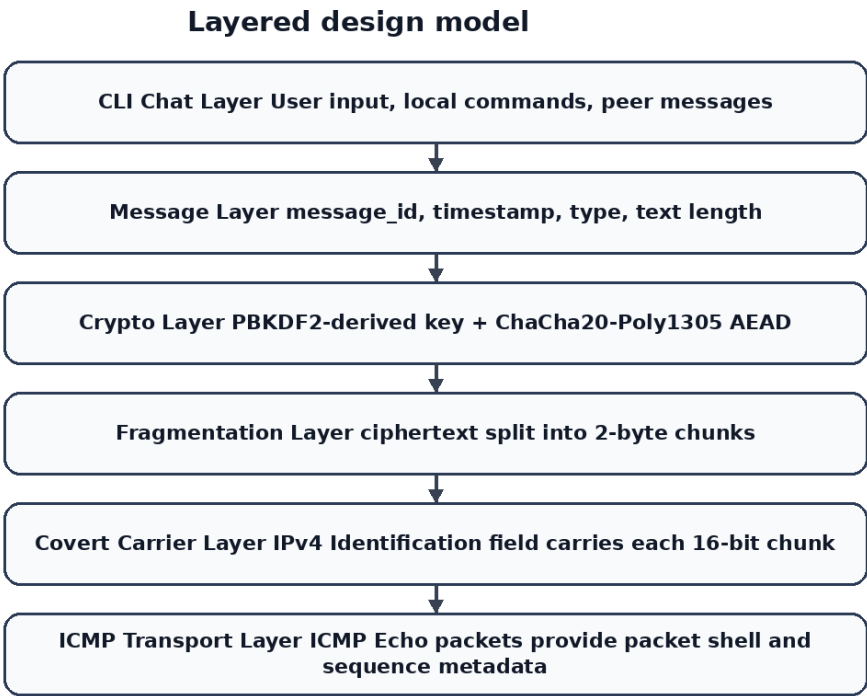
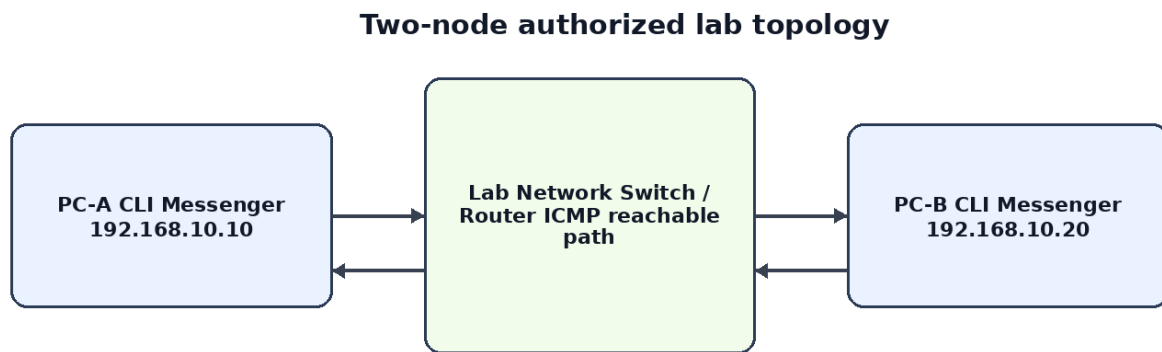


Figure 3 - Layered design model

The project should be implemented as a layered system rather than one large script. The CLI should not directly craft packets, and the cryptographic module should not understand terminal commands. Clean separation lets the same protocol engine support a future GUI.

Module	Responsibility
cli.py	Terminal UI, commands, user input, display formatting
crypto.py	Key derivation, encryption, decryption, nonce generation
protocol.py	Message envelope, packet types, chunking, constants
packet_sender.py	Build and transmit ICMP/IPv4 packets
packet_sniffer.py	Capture packets and pass them to receiver logic
session.py	Session state, message IDs, buffers, statistics
main.py	Startup flow, thread orchestration, shutdown handling

7. Lab Topology and Runtime Model



Both endpoints run the same program: sender thread + receiver/sniffer thread + (

Figure 4 - Two-node runtime model

Each endpoint runs the same program. The program contains a foreground CLI loop and a background receiver/sniffer loop. The CLI loop reads local user input and sends outbound messages. The receiver loop continuously monitors inbound ICMP traffic from the peer and reconstructs inbound messages.

Runtime component	Thread/process role
Main thread	Startup, configuration, command handling, user input loop
Receiver thread	Packet capture, filtering, fragment buffering, decryption trigger
Sender functions	Message envelope creation, encryption, chunking, packet transmission
Status/statistics state	Counters, current peer, current session ID, debug mode

8. Cryptographic Design

8.1 Why ChaCha20-Poly1305

ChaCha20-Poly1305 is an Authenticated Encryption with Associated Data algorithm. It combines the ChaCha20 stream cipher with the Poly1305 authenticator. This is a good fit because the project needs confidentiality, integrity, and authentication without designing a custom cryptographic construction.

- Confidentiality: protects message content.
- Integrity: detects modification of ciphertext fragments.
- Authentication: validates that the receiver has the correct shared key and that the ciphertext/tag pair is valid.
- Simplicity: available through standard Python cryptography libraries.
- Efficiency: performs well on systems without AES-specific CPU acceleration.

8.2 Password-Based Key Derivation

Users should not type raw 32-byte symmetric keys. The CLI should ask for a shared password, then derive a 32-byte key using a password-based key derivation function. PBKDF2-HMAC-SHA256 is simple and widely available;

Argon2id is stronger for password hashing but adds dependency complexity. The first release can use PBKDF2, with a future migration path to Argon2id.

Input	Purpose
Shared password	Human-entered secret known to both endpoints
Salt	Prevents identical passwords from deriving identical keys across contexts
Iterations	Increases brute-force cost
Output length	32 bytes for ChaCha20-Poly1305 key

8.3 Encrypt Once, Then Fragment

The entire internal message envelope should be encrypted once and then fragmented. Encrypting every two-byte fragment separately would be inefficient because AEAD produces authentication overhead for each encryption operation. The correct pipeline is:

1. Create plaintext envelope.
2. Encrypt the full envelope once.
3. Append or retain the authentication tag as part of the ciphertext output.
4. Pad if necessary for an even number of bytes.
5. Split into two-byte chunks.
6. Encode each two-byte chunk into one IPv4-ID value.

9. Packet Format and Field Allocation

9.1 Internal Plaintext Envelope

Field	Size	Description
MAGIC	2 bytes	Constant marker such as NS after decryption.
VERSION	1 byte	Protocol version, initially 0x01.
MSG_TYPE	1 byte	Chat, control, future file/control types.
MESSAGE_ID	4 bytes	Unique sender-side message number.
TIMESTAMP	4 bytes	Unix timestamp or monotonic timestamp representation.
TEXT_LEN	2 bytes	Length of UTF-8 text body.
TEXT	Variable	User message bytes.

9.2 Outer Packet Mapping

Outer field	Usage
IPv4 source/destination	Normal endpoint addressing.
IPv4 Identification	Hidden two-byte ciphertext fragment.
ICMP Identifier	Session ID, stable during chat session.
ICMP Sequence Number	Packet role and ordering.
ICMP Payload	Fixed lab payload; not the secret content.

9.3 Packet Types

Packet type	ICMP sequence convention	IPv4-ID meaning
START	0	Message ID lower 16 bits or start marker.
DATA	1 to N	Two-byte encrypted fragment.
END	65534	Total number of chunks.
ACK	65535 with ACK convention	Message accepted.
NACK	65535 with NACK convention	Message incomplete or failed; first release may retransmit full message.

Because Version 2 intentionally gives the IPv4-ID field to ciphertext fragments, control information must be carried through ICMP Identifier, ICMP Sequence, packet direction, and minimal control conventions rather than through the secret carrier field.

10. Message Lifecycle: Sender to Receiver

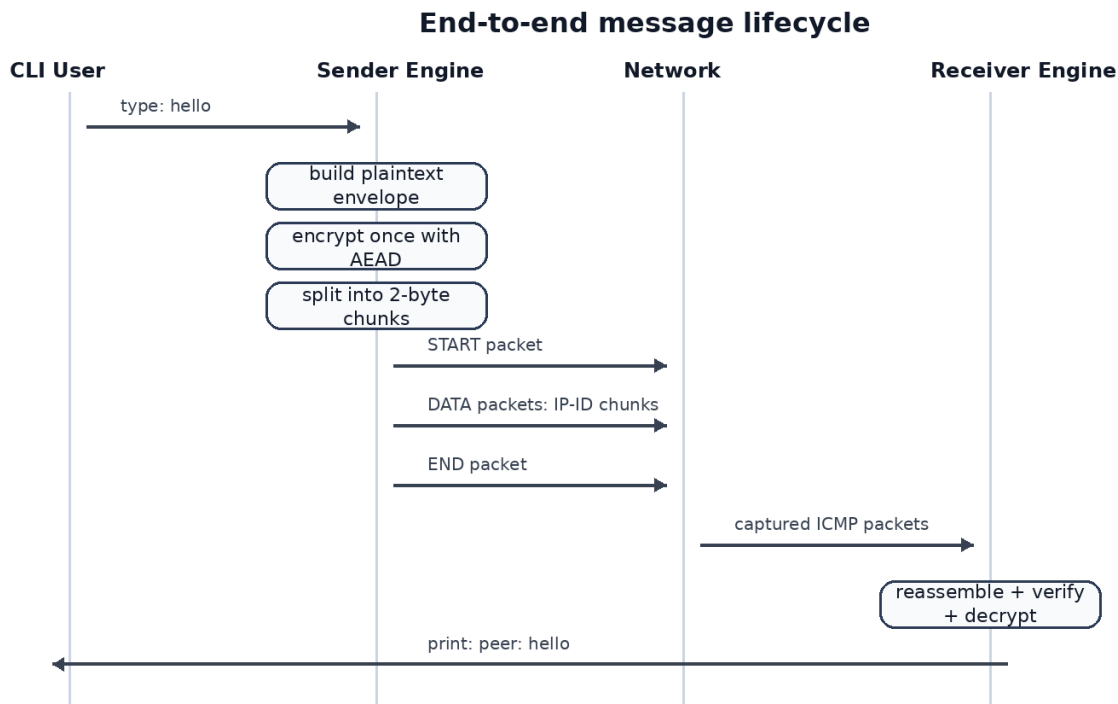


Figure 5 - End-to-end lifecycle

10.1 Sender Pipeline

7. User enters a message in the CLI prompt.
8. The CLI checks whether the input begins with @. If yes, it is processed as a command. If no, it becomes a chat message.
9. The message layer creates a plaintext envelope containing magic, version, message type, message ID, timestamp, text length, and text.
10. The crypto layer derives or retrieves the current session key, creates a nonce, and encrypts the plaintext envelope using ChaCha20-Poly1305.
11. The fragmentation layer splits the encrypted result into two-byte chunks. If the result length is odd, it adds one padding byte and records enough metadata to remove it after decryption or envelope parsing.
12. The packet sender transmits a START packet, then DATA packets, then an END packet.
13. The sender waits for ACK or NACK within a timeout window. On NACK or timeout, the first implementation can retransmit the full message.

10.2 Receiver Pipeline

14. The sniffer captures inbound ICMP packets.
15. The receiver filters by source IP and expected ICMP Identifier/session ID.
16. If the ICMP Sequence is START, a message buffer is created.
17. If the sequence is DATA, the receiver stores IPv4-ID value as the chunk for that sequence number.

18. If the sequence is END, the receiver checks whether all expected chunk numbers are present.
19. If chunks are missing, the receiver sends NACK or waits for retransmission.
20. If all chunks are present, it sorts by sequence number, concatenates chunks, and passes the ciphertext to the crypto layer.
21. The crypto layer verifies the authentication tag and decrypts. If verification fails, the message is dropped and logged.
22. After successful decryption, the envelope is parsed and the text is printed as peer output.

11. CLI Messenger Concept and User Experience

```
=====
Network Steganography Messenger v2
ICMP / IPv4-ID / ChaCha20-Poly1305
=====

Enter destination IP address: 192.168.10.20
Enter shared password: *****
Checking connectivity...
[OK] ICMP reachability confirmed.

Type @help for commands.

me: hello Ahmad
[12:30:01] sending message_id=42 chunks=31
[12:30:02] delivered

peer: hello there
me: @status
session=0x41AF peer=192.168.10.20 tx=1 rx=1 debug=off
me:
```

Figure 6 - CLI messenger startup and chat concept

The first user interface should be intentionally simple. The user launches the program, enters the peer IP address, enters a shared password, runs a connectivity check, and enters an interactive chat loop. The UI should feel like a minimal terminal version of a two-person messaging application.

CLI phase	User experience	Internal action
Startup	Welcome banner	Initialize configuration and logging
Peer selection	Enter destination IP	Validate IPv4 format and store peer address
Password entry	Enter shared password hidden from terminal	Derive encryption key
Connectivity check	Checking connectivity... OK/Warning	Send normal ICMP probe or packet-level test
Chat loop	me: prompt	Process commands or send chat messages
Inbound display	peer: message	Receiver thread prints completed messages

11.1 Example CLI Session

Network Steganography Messenger v2
ICMP / IPv4-ID / ChaCha20-Poly1305

```
Enter destination IP address: 192.168.10.20
Enter shared password: *****
Checking connectivity...
[OK] ICMP reachability confirmed.
```

Type @help for commands.

```
me: hello
[12:30:01] sending message_id=42 chunks=31
[12:30:02] delivered
peer: hello there
me: @status
session=0x41AF peer=192.168.10.20 tx=1 rx=1 debug=off
me: @finish
Closing session... done.
```

12. Control Commands and Session Management

Command	Purpose	Expected behavior
@help	Show available commands	Print command list and short descriptions.
@finish	Exit the program	Stop sniffer thread, close sockets, print summary.
@changedstip	Change peer destination IP	Validate new IP, run optional connectivity check.
@password	Change shared password	Re-derive key, reset or rotate session parameters.
@status	Show session state	Peer IP, session ID, counters, debug mode, last ACK.
@check	Run connectivity check	Send normal ICMP reachability test.
@debug on/off	Toggle verbose technical logs	Show packet IDs, chunks, state transitions when enabled.
@stats	Show counters	TX/RX messages, packets, retransmits, failures.
@clear	Clear terminal	Keep state, clear screen.

12.1 Session IDs and Message IDs

The ICMP Identifier field should represent a session ID. The session ID allows both nodes to distinguish messenger packets from unrelated ICMP packets. Each sent chat message should also receive a message ID. Because the IPv4-ID field is reserved for hidden ciphertext fragments, message identity must be handled through a combination of START packet convention, local state, and session tracking.

13. Reliability Model: START, DATA, END, ACK, NACK

ICMP does not provide delivery guarantees. Version 2 therefore needs a simple reliability layer. The reliability layer should not attempt to become full TCP; it only needs enough logic to deliver short chat messages in a lab network.

Mechanism	First release behavior	Future improvement
START	Create receive buffer	Add stronger message metadata and anti-replay windows
DATA	Store chunk by sequence number	Selective retransmission for missing chunks
END	Declare total chunk count	Include checksum over ordered ciphertext before AEAD verification
ACK	Mark message delivered	ACK specific message ID
NACK	Retransmit full message	NACK specific missing chunk ranges
Timeout	Retry or mark failed	Adaptive timeout based on RTT

13.1 Recommended First-Release Reliability

- Use whole-message retransmission first; it is easier to debug.
- Keep a sent-message cache for the last few messages.
- Expire incomplete receive buffers after a timeout.
- Drop duplicate chunks but count them in statistics.
- Log authentication failures separately from missing chunks.

14. Receiver State Machine and Reassembly

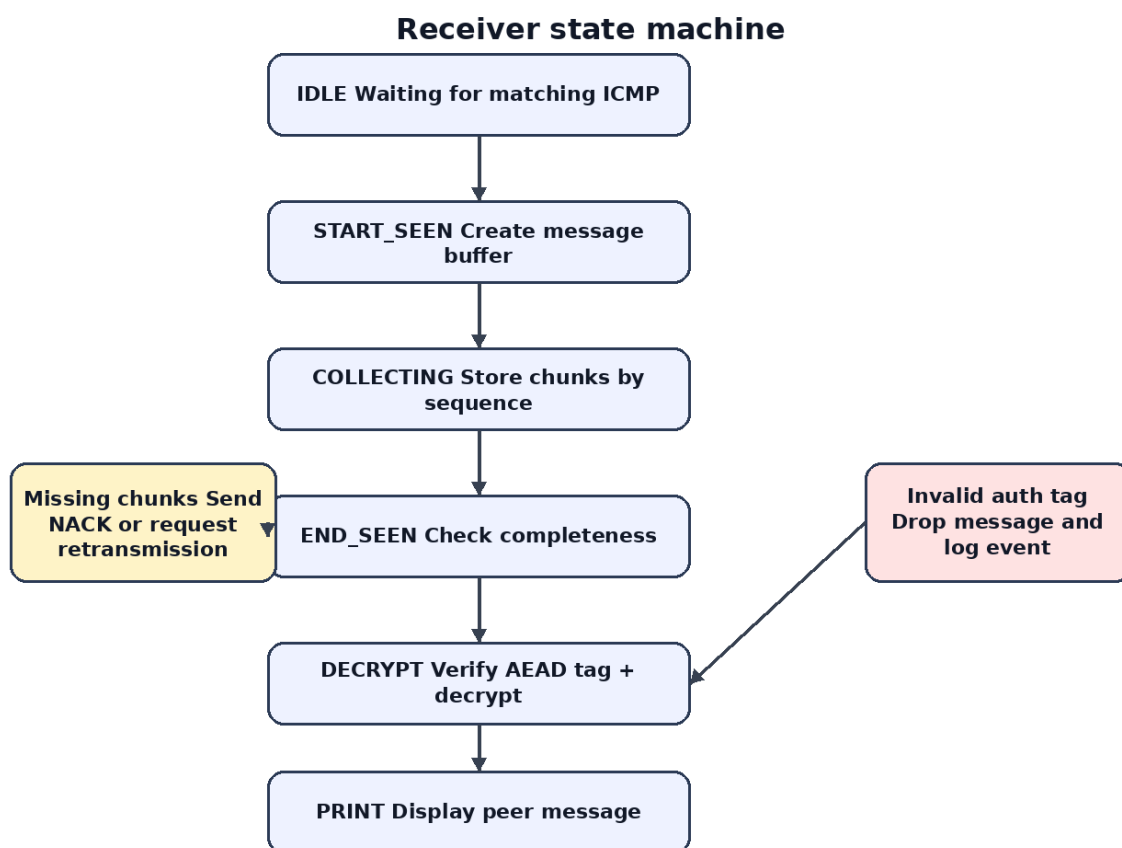


Figure 7 - Receiver state machine

The receiver should be strict. It should not attempt to decrypt until it has observed an END packet and all expected DATA chunks. It should also avoid blocking the CLI while waiting for incomplete messages. A background thread and message buffer map solve this cleanly.

Receive buffer concept:

```
buffers = {
  session_id: {
    current_message_id: {
      "chunks": {1: b"..", 2: b"..", 3: b".."},
      "expected_total": 31,
      "start_time": 1710000000,
      "peer_ip": "192.168.10.20"
    }
  }
}
```

When all chunks from 1..expected_total exist:

```
ciphertext = chunks[1] + chunks[2] + ... + chunks[N]
plaintext = decrypt_and_verify(ciphertext)
print("peer:", parsed_text)
```

15. Operational Considerations and Failure Modes

Failure mode	Cause	Handling
No connectivity	ICMP blocked, routing issue, firewall	Warn user, allow destination change, run @check.
IP-ID rewritten	NAT, tunnel, firewall normalization, OS behavior	Detect through test packet, document as unsupported path.
Packet loss	Network drops, rate limiting	ACK/NACK and retransmission.
Out-of-order packets	Normal network behavior	Store by ICMP sequence number.
Authentication failure	Wrong password, corrupted chunks, altered packets	Drop message and log failure.
Duplicate packets	Retransmission or capture artifact	Ignore duplicate chunk numbers.
Odd ciphertext length	Two-byte carrier alignment issue	Pad at fragmentation layer and remove after reassembly/envelope parsing.

15.1 Preflight Tests Before Chat

- 23. Validate destination IPv4 address format.
- 24. Run normal ICMP reachability test.
- 25. Run a field-preservation test in lab: send a known IPv4-ID and confirm receiver observes the same value.
- 26. Confirm both endpoints use the same password and compatible protocol version.
- 27. Enable debug mode during first tests, then disable it for normal demonstration.

16. Defensive Analysis and Blue-Team Validation

Defensive validation pipeline

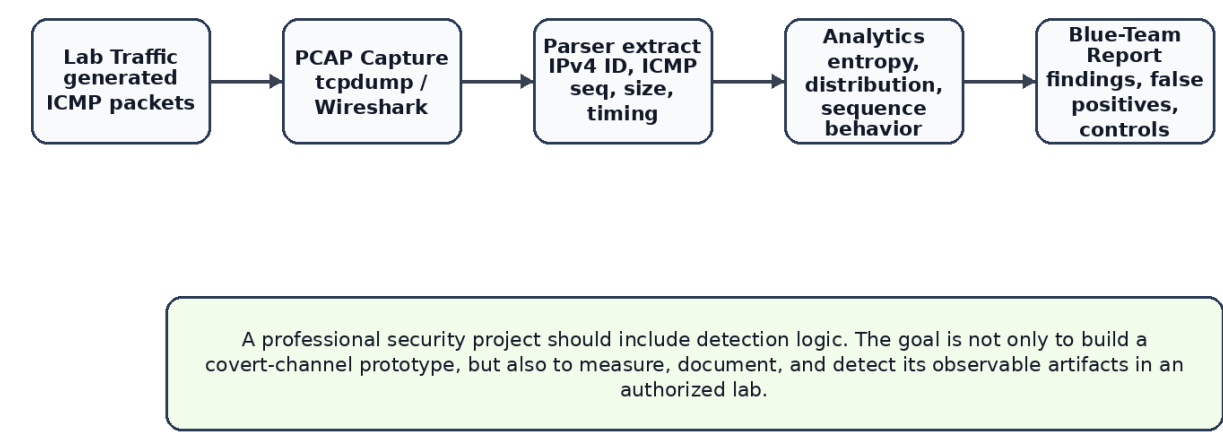


Figure 8 - Defensive validation pipeline

A high-quality security project should include both construction and detection. The blue-team side should capture the lab traffic and analyze observable artifacts. The point is not merely to build a messenger; it is to understand what defenders can see and how protocol behavior changes when a covert channel exists.

Observable	Defensive question
IPv4-ID distribution	Does it look sequential, random, constant, or structured?
ICMP sequence behavior	Are sequence numbers consistent with normal ping behavior?
Packet rate	Are ICMP packets sent in bursts during chat messages?
Packet sizes	Are payload lengths fixed or unusual?
Request/reply symmetry	Do Echo Replies match Echo Requests normally?
Session duration	Is there long-lived ICMP communication between two hosts?
Authentication failures	Can tampering or wrong-password tests be observed as failed decrypt events?

16.1 Suggested Blue-Team Deliverables

- PCAP files for normal ping baseline and messenger traffic.
- Wireshark display filters and screenshots for IPv4-ID and ICMP sequence behavior.
- A small analysis report comparing baseline vs project traffic.
- Optional detector script that extracts IPv4-ID values and plots distribution over time.
- Recommended controls: ICMP rate limiting, anomaly detection, egress filtering, and policy-based monitoring for long ICMP sessions.

17. Implementation Roadmap

Phase	Deliverable	Notes
Phase 1	CLI skeleton	Welcome screen, destination IP, password prompt, @help, @finish.
Phase 2	Connectivity check	Normal ICMP reachability test.
Phase 3	Crypto module	PBKDF2 key derivation and ChaCha20-Poly1305 encrypt/decrypt tests.
Phase 4	Message envelope	Pack/unpack MAGIC, VERSION, MSG_TYPE, MESSAGE_ID, TIMESTAMP, TEXT_LEN, TEXT.
Phase 5	Fragmentation	Split ciphertext into two-byte chunks and reassemble.
Phase 6	Packet sender	Craft ICMP Echo packets with IPv4-ID set to chunk value.
Phase 7	Packet receiver	Sniff ICMP, filter peer/session, collect chunks.
Phase 8	Reliability	START/DATA/END/ACK and full-message retransmission.
Phase 9	Debug/statistics	@debug, @status, @stats, technical logs.
Phase 10	Blue-team validation	PCAP capture, field analysis, detection notes.

17.1 First Coding Milestone

The first coding milestone should not immediately send real covert packets. It should prove the internal pipeline locally: message envelope -> encryption -> two-byte fragmentation -> reassembly -> decryption -> parsed text. Once that works, packet transmission can be added safely and debugged separately.

18. References

- RFC 791 - Internet Protocol. RFC Editor. <https://www.rfc-editor.org/info/rfc791/>
- RFC 792 - Internet Control Message Protocol. IETF Datatracker. <https://datatracker.ietf.org/doc/html/rfc792>

- RFC 6864 - Updated Specification of the IPv4 ID Field. IETF Datatracker.
<https://datatracker.ietf.org/doc/html/rfc6864>
- RFC 8439 - ChaCha20 and Poly1305 for IETF Protocols. IETF Datatracker.
<https://datatracker.ietf.org/doc/html/rfc8439>
- RFC 8018 - PKCS #5: Password-Based Cryptography Specification Version 2.1. IETF Datatracker.
<https://datatracker.ietf.org/doc/html/rfc8018>
- IANA ICMP Parameters Registry. <https://www.iana.org/assignments/icmp-parameters>